

A P S 27.2 Studio di caso: Monopoly

Casi d'uso, SSD, contratti

- niente di nuovo

Modello di dominio

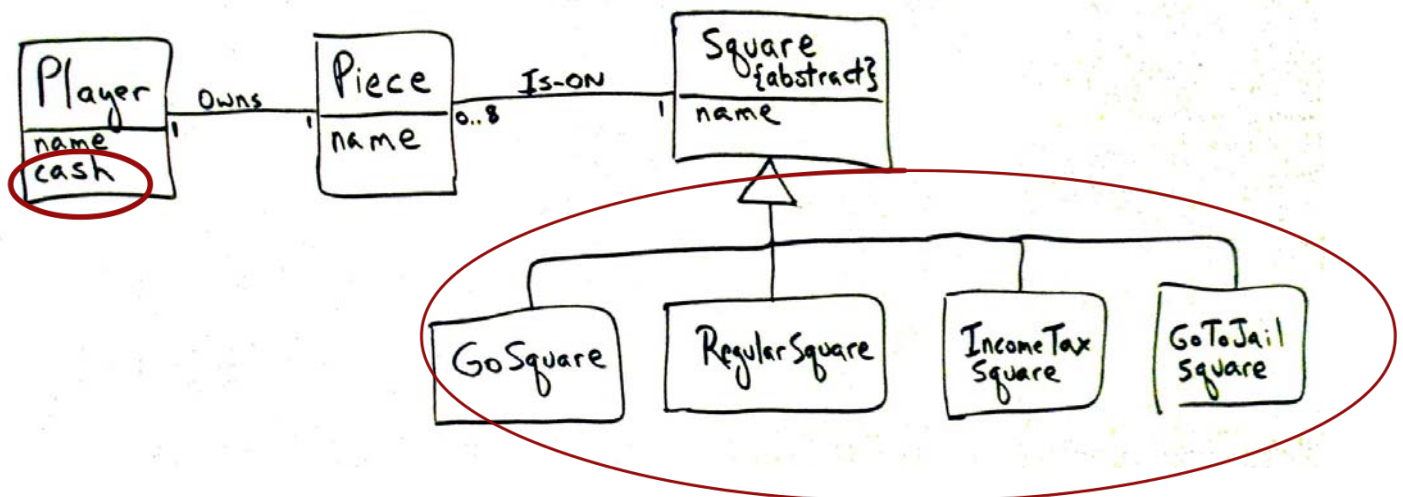
- importo di denaro posseduto da ciascun giocatore
- alcune tipologie di caselle – rappresentate mediante gerarchie di generalizzazione-specializzazione ^(via, prigione, passa)

15

Rapido aggiornamento dell'analisi (e del progetto)

Luca Cabibbo – A·P·S

A P S Cambiamenti nel modello di dominio



16

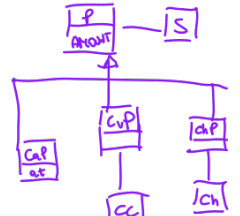
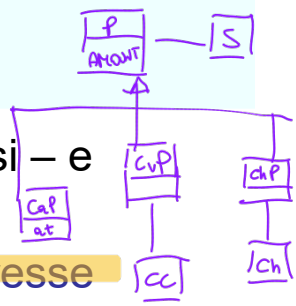
Rapido aggiornamento dell'analisi (e del progetto)

Luca Cabibbo – A·P·S

A P S Quando definire una sottoclasse

Motivazioni valide per partizionare una classe in sottoclassi – e mostrare una sottoclasse

- la sottoclasse ha proprietà strutturali aggiuntive di interesse
 - la sottoclasse ha attributi aggiuntivi di interesse
 - la sottoclasse ha associazioni aggiuntive di interesse
- la sottoclasse è caratterizzata da un comportamento diverso
 - il concetto della sottoclasse viene fatto funzionare, viene gestito, reagisce o viene manipolato in modo diverso dalla superclasse o dalle altre sottoclassi, secondo modalità di interesse
 - il concetto della sottoclasse rappresenta qualcosa di animato che si comporta in modo diverso dalla superclasse o dalle altre sottoclassi, secondo modalità di interesse



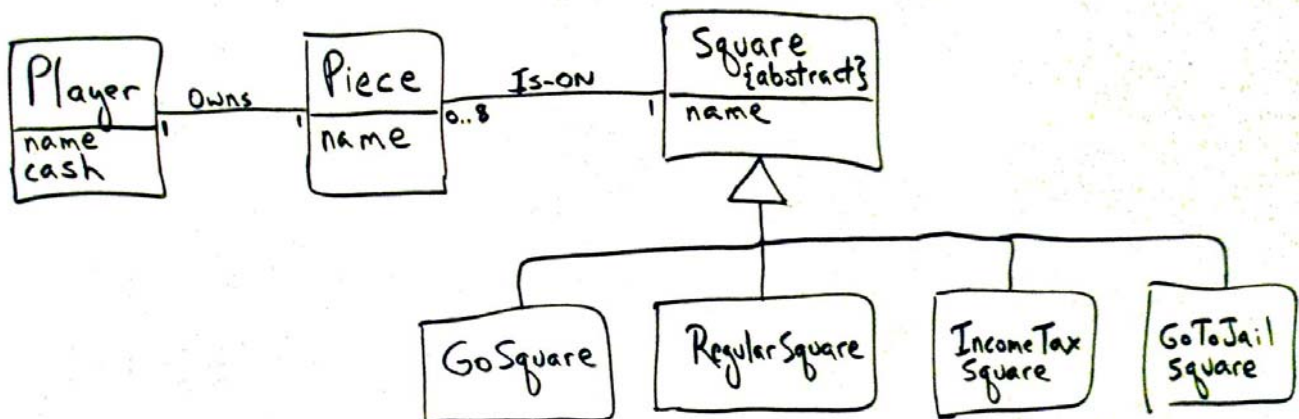
L'autorizzazione di ogni pagamento è differente CONTANTI ≠ VISA

17

Ulteriore modellazione di dominio

Luca Cabibbo – A·P·S

A P S Esempio



NON HANNO PROP. STRUTTURALI AGGIUNTIVE
MA IL COMPORTAMENTO DEL GIOCATTORE SU
UNA DI QUELLE CASE È DIVERSO DALL'ALTRA

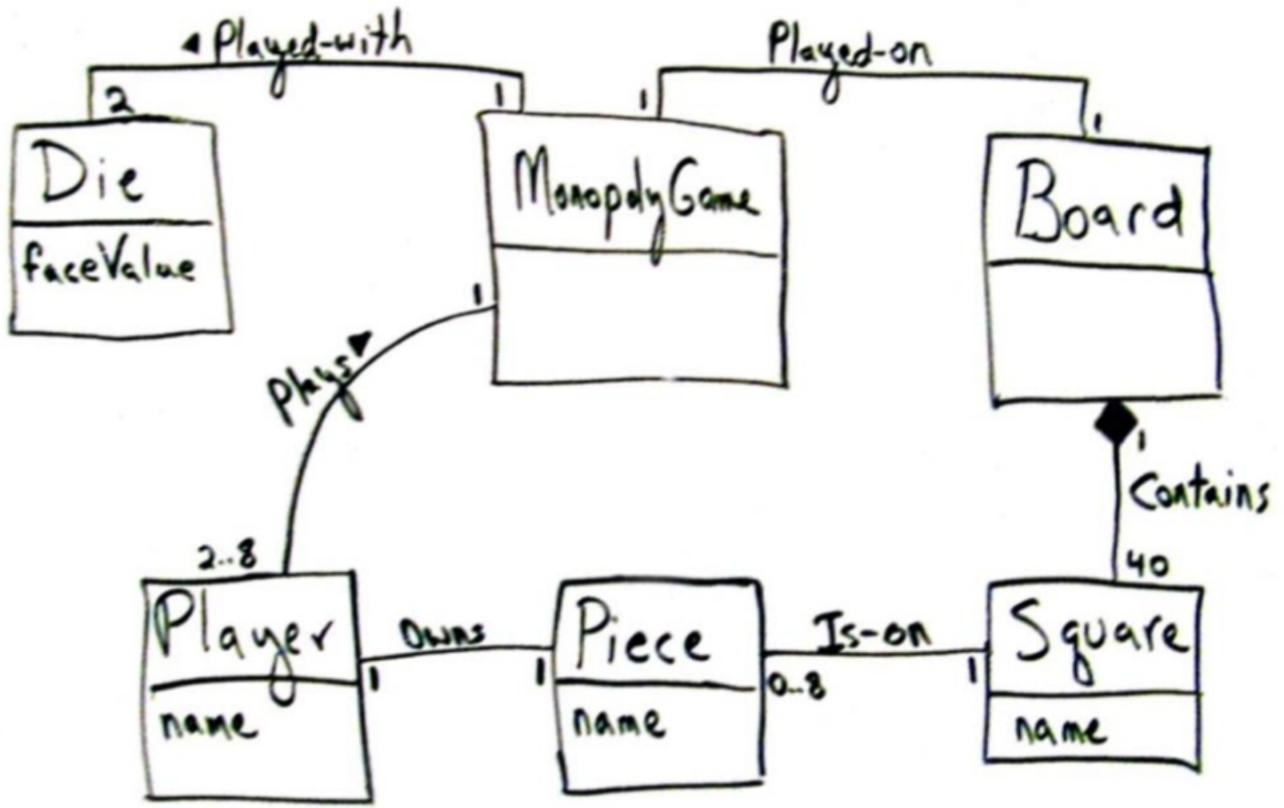
18

Ulteriore modellazione di dominio

Luca Cabibbo – A·P·S

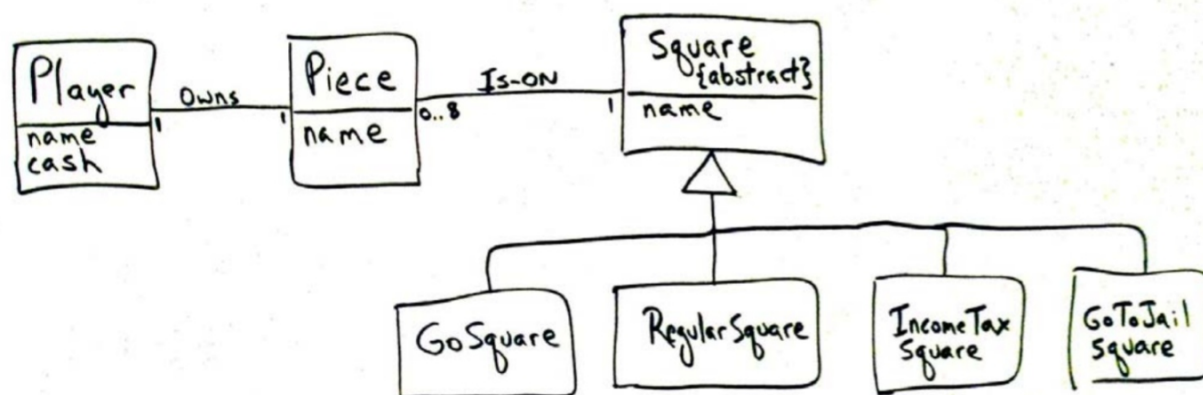
A P S 34.17 Raffinamenti per Monopoly

Modello di dominio per l'iterazione 1



A P S Raffinamenti per Monopoly

Raffinamenti al modello di dominio per l'iterazione 2



Monopoly

I requisiti aggiuntivi per la seconda iterazione dell'applicazione Monopoly comprendono i seguenti.

- Di nuovo, implementare uno scenario di base del caso d'uso Gioca una Partita a Monopoly: i giocatori si spostano lungo le caselle del tabellone. E come prima, il gioco è eseguito come una simulazione, senza bisogno di input dell'utente, a parte il numero di giocatori. Tuttavia, nell'iterazione 2 si applicano le regole su alcune delle caselle speciali, descritte nei punti seguenti...

- Ogni giocatore riceve all'inizio del gioco 1500 dollari. Si fa l'ipotesi che sia disponibile una quantità illimitata di denaro.

- Quando un giocatore finisce sulla casella Go (Via!), riceve 200 dollari.

- Quando un giocatore finisce sulla casella Go-To-Jail (In prigione!) si sposta sulla casella Jail (Prigione). Tuttavia, diversamente dalle regole complete, ne esce facilmente. Al turno successivo, tira semplicemente i dadi e si sposta come indicato dal totale dei dadi.

- Quando un giocatore finisce sulla casella IncomeTax (Tassa patrimoniale), paga il minimo tra 200 dollari e il 10% di ciò che possiede.

Ecco i requisiti per il Monopoly scelti per questa iterazione, scritti come storie utente.

- Se un giocatore finisce sulla casella del via, allora riceve 200 dollari.

- Se un giocatore finisce sulla casella "tassa patrimoniale", allora paga la tassa patrimoniale.

- Se un giocatore finisce sulla casella "vai in prigione", allora va in prigione.

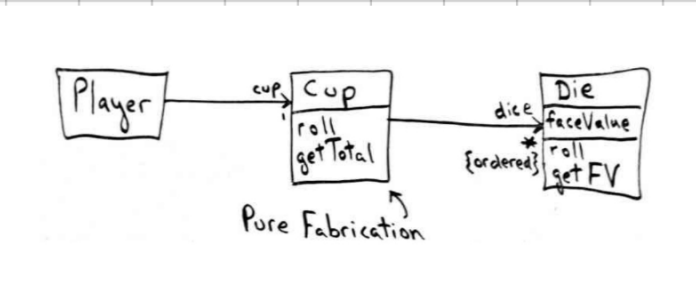


Figura 28.2 DCD per Cup.

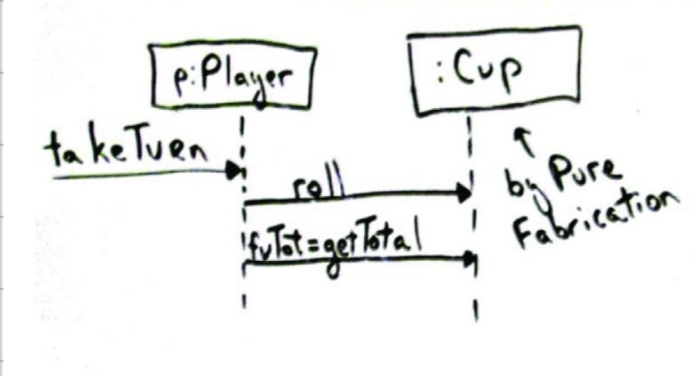


Figura 28.3 Utilizzo di Cup nel gioco Monopoly.

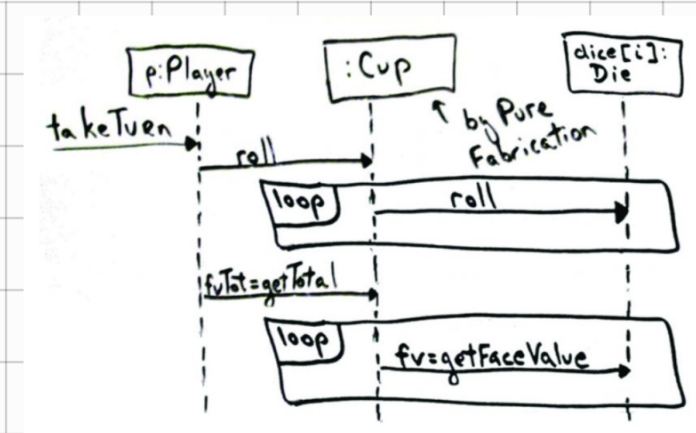


Figura 28.4 Un maggiore dettaglio circa l'utilizzo di Cup.

Il problema riguarda come il gioco del Monopoly gestisce il lancio dei dadi. Inizialmente, è il "Giocatore" (Player) a tirare i dadi e sommarne il totale.

Questa soluzione presenta dei problemi:

- Il modo in cui i dadi vengono gestiti non è riutilizzabile facilmente per altri giochi, perché è specifico per il "Giocatore" del Monopoly.
- Non si può semplicemente chiedere il totale dei dadi senza doverli tirare di nuovo.

• Il "Giocatore" è troppo legato ai dadi: sa quanti sono, come usarli e come calcolare il totale. Se qualcosa sui dadi cambia, anche la classe "Giocatore" deve cambiare.

La soluzione proposta è usare un concetto chiamato "Pure Fabrication". Questo significa inventare un nuovo oggetto che non esiste direttamente nel mondo reale del Monopoly (come un bussolotto) per gestire i dadi.

Questo nuovo oggetto si chiama "Cup" (Bussolotto).

- Il "Cup" contiene i dadi.
- È il "Cup" che ha la responsabilità di tirare i dadi e calcolarne il totale.

• Quando si dice al "Cup" di lanciare i dadi, è lui che dice a tutti i dadi al suo interno di "rotolare".

In pratica, si sposta la responsabilità di gestire i dadi dal "Giocatore" a un nuovo oggetto "Cup", rendendo il sistema più pulito, riutilizzabile e meno dipendente

Per rendere il codice più pulito, flessibile e riutilizzabile:

1. Gestione del Lancio dei Dadi (con l'oggetto "Cup"):

◦ Problema Iniziale: Il "Giocatore" (Player) sapeva troppi dettagli sui dadi (quanti sono, come usarli, come calcolare il totale). Questo creava un "accoppiamento indesiderato" tra il Giocatore e i dadi. Se i dadi o il loro uso fossero cambiati, anche la classe del Giocatore avrebbe dovuto cambiare.

◦ Soluzione: Viene introdotto un nuovo oggetto chiamato "Cup" (Bussolotto) [previous summary]. Questo è un esempio di "Pure Fabrication" (inventare un oggetto che non ha una controparte fisica diretta ma serve a scopi software) [previous summary].

◦ Ruolo della "Cup": La Cup ha la responsabilità esclusiva di gestire i dadi (tirarli e calcolarne il totale) [previous summary]. Questo supporta il principio di "Indirection" (Indirezione), che significa interporre un oggetto (la Cup) per ridurre la dipendenza diretta tra altri oggetti (Player e Die). In questo modo, se qualcosa cambia sui dadi, si modifica solo la Cup e non il Player.

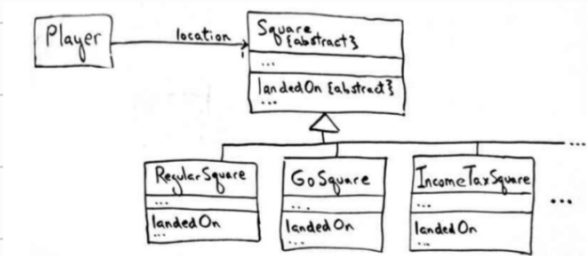


Figura 28.5 Applicazione di Polymorphism al problema Monopoly.

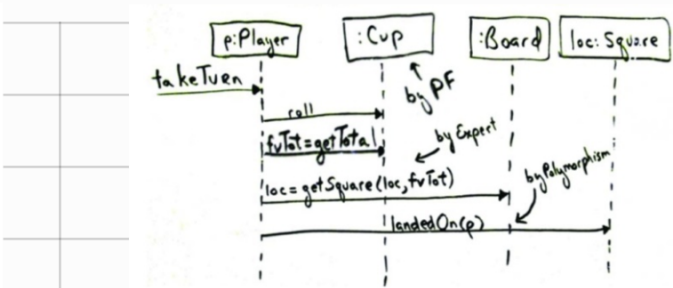


Figura 28.6 Applicazione di Polymorphism.

Il problema principale nel gioco del Monopoly riguarda come gestire le diverse azioni che accadono quando un giocatore finisce su caselle diverse.

* **Il Problema del "Brutto Progetto:** Una soluzione sbagliata (un "brutto progetto") sarebbe usare un sistema con molte condizioni (`SWITCH` o `case`) che controllano il tipo di casella e poi eseguono l'azione specifica. Questo rende il codice rigido e difficile da modificare.

* **La Soluzione Migliore: Il Polimorfismo:**

- * Il suggerimento è di usare il concetto di **Polimorfismo**.
- * Questo significa creare una **funzione** (o "operazione polimorfa") comune chiamata, ad esempio, `landedOn` ("finisce su").
- * Ogni tipo specifico di casella (es. "casella Via", "casella Tassa sul Reddito", "casella Prigione", "casella Regolare") avrà la **propria versione di questa funzione** `landedOn`, che descrive esattamente cosa succede quando un giocatore ci finisce sopra.
- * Nella classe generale "Casella" (la "superclasse"), questa funzione `landedOn` è dichiarata come **astratta**, il che significa che ogni casella specifica **deve fornire la sua implementazione**.

* **Chi Invia il Messaggio? È il Giocatore:**

- * La nota spiega che è il "Giocatore" (Player) a dover dire alla casella "Sono finito su di te, esegui la tua azione!".
- * Questo è perché il Giocatore sa già su quale casella si trova. Questa scelta segue principi di buon design software come "Accoppiamento Basso" (ridurre le dipendenze tra le parti del codice) e "Esperto di Informazioni" (la responsabilità va a chi ha le informazioni necessarie).
- * Il messaggio `landedOn` viene inviato dalla classe `Player` alla `Square` corretta. Spesso, il Giocatore (`Player`) viene passato come parametro alla funzione `landedOn` della casella, così la casella può interagire direttamente con il Giocatore (es. dargli soldi o fargli pagare).

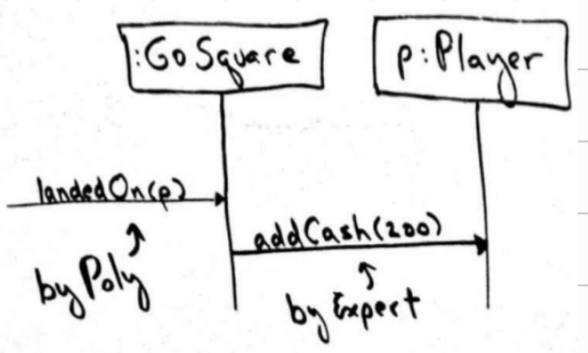


Figura 28.7 Il caso GoSquare.

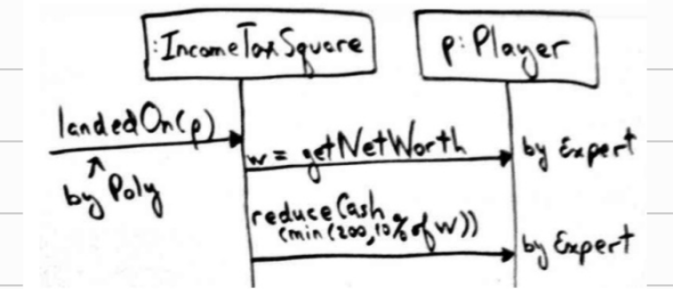


Figura 28.8 Il caso IncomeTaxSquare.

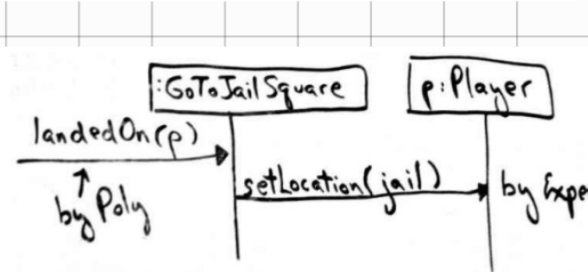


Figura 28.9 Il caso GoToJailSquare.

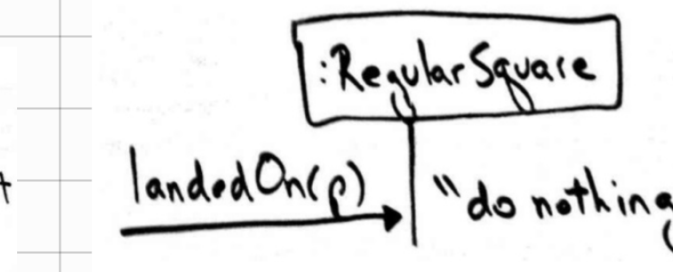


Figura 28.10 Il caso RegularSquare.

* **Esempi delle Azioni `landedOn`:**

- * **GoSquare (casella Via):** Dice al Giocatore di **aggiungere contanti** (es. 200 dollari).
- * **IncomeTaxSquare (casella Tassa):** Chiede al Giocatore di calcolare il 10% del suo patrimonio e di pagare quella somma.
- * **GoToJailSquare (casella Vai in Prigione):** Dice al Giocatore di **cambiare la sua posizione** e andare in prigione.
- * **RegularSquare (casella Normale):** Non succede nulla. La sua funzione `landedOn` è vuota, ma esiste per mantenere la coerenza del polimorfismo ed evitare logiche speciali.

* **Migliorare la Relazione "Giocatore-Casella":**

- * Inizialmente, la pedina del gioco ("Piece") conosceva la posizione della casella, e il Giocatore doveva chiederla alla pedina. Questo era un design debole per l'accoppiamento.
- * è il **Giocatore stesso che dovrebbe conoscere direttamente la casella in cui si trova**.
- * Questo rende il sistema più pulito e efficiente, perché il Giocatore collabora continuamente con la sua Casella. La pedina fisica ("Piece") potrebbe persino non essere necessaria nel modello software, dato che il Giocatore può svolgere quel ruolo.

2. Gestione delle Azioni sulle Caselle (con l'operazione landedOn):

- Problema Iniziale: Un modo sbagliato di gestire le azioni delle caselle sarebbe usare lunghe liste di condizioni (SWITCH o case) per ogni tipo di casella [previous summary]. Questo rende il codice rigido e difficile da modificare [previous summary].
- Soluzione: Si usa il Polimorfismo [previous summary]. Significa creare una funzione comune (landedOn) nella classe generica "Casella" (Square) [previous summary]. Ogni tipo specifico di casella (es. "Via", "Tassa", "Prigione", "Casella normale") ha la propria versione di questa funzione landedOn che descrive cosa succede quando un giocatore ci finisce sopra [previous summary].
- Chi Invia il Messaggio?: È il "Giocatore" (Player) che dice alla casella su cui è finito di eseguire la sua azione (landedOn) [previous summary]. Il Player è l'oggetto più adatto a fare ciò perché sa già dove si trova [previous summary].
- Pattern di Design: Questa implementazione della funzione landedOn è un'applicazione del Design Pattern "Strategy".
 - Il Player è l'oggetto "contesto".
 - I "comportamenti" specifici che avvengono quando il giocatore finisce su una casella (ovvero le diverse implementazioni di landedOn) sono le "strategie".
 - I diversi tipi di "Casella" (Square) sono gli "oggetti strategia" che contengono l'implementazione specifica di ogni strategia.
- Esempi delle Azioni: La landedOn di una casella "Via" aggiunge soldi, quella della "Tassa" fa pagare, quella della "Prigione" sposta il giocatore, e quella di una casella "Normale" non fa nulla (è vuota) [previous summary].
- Relazione Giocatore-Casella: Si preferisce che il Giocatore conosca direttamente la Casella in cui si trova, per una migliore collaborazione e un design più pulito [previous summary].

30.2 Monopoly

I requisiti dell'iterazione 3 comprendono i seguenti.

Implementare di nuovo uno scenario di base del caso d'uso Gioca una Partita a Monopoly: i giocatori si spostano sulle caselle del tabellone. Come in precedenza, il gioco viene eseguito come una simulazione che non richiede alcun input dall'utente, a parte il numero di giocatori. Nell'iterazione 3 si applica tuttavia un insieme più completo di regole, descritto nei seguenti punti.

Ci sono ora delle caselle Lot (Proprietà), Railroad (Ferrovia) e Utility (Società). Quando un giocatore finisce su una di queste caselle, si applica la logica descritta nei punti seguenti.

Se la casella Lot, Railroad o Utility non è posseduta da nessuno, il giocatore che ci è finito sopra può acquistarla. In questo caso, il prezzo di questa casella viene detratto dal denaro del giocatore e questi diventa il proprietario.

Il prezzo viene stabilito quando inizia il gioco, ma è arbitrario; per esempio, si possono utilizzare i prezzi ufficiali di Monopoly.

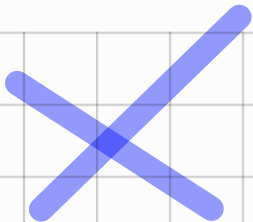
Se la casella Lot, Railroad o Utility è di proprietà del giocatore che ci è finito sopra, non succede niente.

Se la casella Lot, Railroad o Utility è di proprietà di un giocatore diverso da quello che ci è finito sopra, quest'ultimo deve pagare l'affitto al proprietario. L'affitto viene calcolato come segue.

L'affitto per una Lot è pari all'indice della sua posizione, in dollari; per esempio, se è la casella in posizione 5, saranno 5 dollari.

L'affitto per una Railroad è pari a 25 volte il numero di Railroad possedute dal proprietario; per esempio se possiede 3 Railroad, saranno 75 dollari.

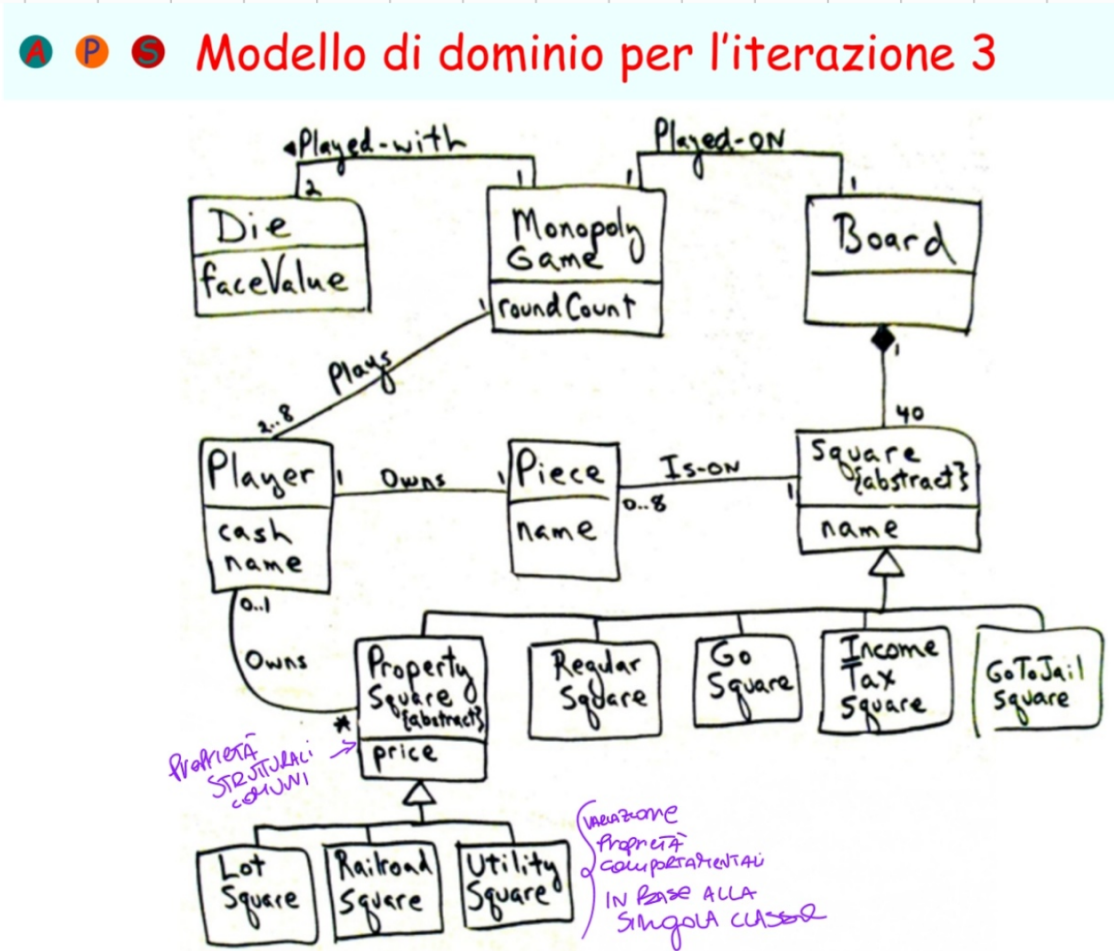
L'affitto per una Utility è pari a 4 volte il totale del lancio dei dadi che ha portato il giocatore su quella casella (non bisogna tirare i dadi di nuovo).



Ecco i requisiti del Monopoly per questa iterazione, scritti come storie utente.

-Se un giocatore finisce su una casella proprietà che non ha un proprietario, allora può acquistare quella casella proprietà.

-Se un giocatore finisce su una casella proprietà il cui proprietario è un altro giocatore, allora gli deve pagare un affitto.



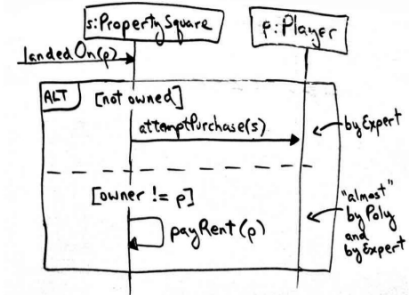


Figura 39.27 Giocatore che finisce su una PropertySquare.

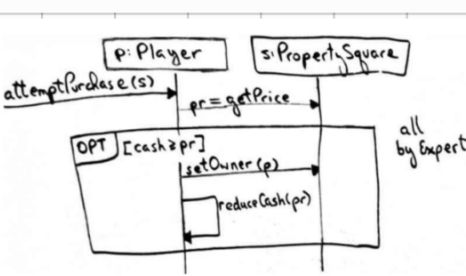


Figura 39.28 Tentativo di acquistare una proprietà.

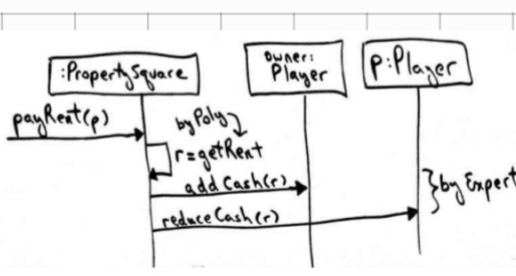


Figura 39.29 Pagamento dell'affitto.

come vengono gestite le caselle "proprietà" (Lot, Railroad, Utility) nel Monopoly e introducono un nuovo approccio di design per rendere il codice più efficiente e flessibile.

1.
- Le Nuove Regole per le Caselle Proprietà:
- Sono state aggiunte caselle di tipo "Lot" (Proprietà), "Railroad" (Ferrovia) e "Utility" (Società).

◦ Quando un giocatore finisce su una di queste caselle:

▪ Se la casella non è posseduta, il giocatore può acquistarla. Il prezzo viene detratto dal suo denaro.

▪ Se la casella è già di proprietà del giocatore stesso, non succede niente.

▪ Se la casella è di proprietà di un altro giocatore, il giocatore che vi è finito sopra deve pagare un affitto.

• L'affitto per una "Lot" dipende dalla sua posizione (es. posizione 5, 5 dollari).

• L'affitto per una "Railroad" è 25 volte il numero di Ferrovie possedute dal proprietario.

• L'affitto per una "Utility" è 4 volte il totale del lancio dei dadi che ha portato il giocatore su quella casella (senza ritirare i dadi).
2.
- Il Problema di Design per le Caselle Proprietà:
- Già in precedenza avevamo visto che il "Giocatore" (Player) invia un messaggio landedOn alla casella su cui è finito per farle eseguire la sua azione, utilizzando il Polimorfismo.

◦ Il problema adesso è che tutte le caselle proprietà hanno un comportamento landedOn molto simile (controllare proprietà, comprare, pagare affitto), ma il calcolo dell'affitto è diverso per ogni tipo di proprietà (Lot, Railroad, Utility).

◦ Una soluzione "cattiva" sarebbe duplicare tutto l'algoritmo in ogni sottoclasse di PropertySquare (Lot, Railroad, Utility), ma questo porterebbe a ripetizioni di codice.

3.
- La Soluzione: Il Design Pattern "Template Method":
- Questo pattern è usato per gestire algoritmi che hanno la stessa struttura generale ma variano in alcuni passi.

◦ Come funziona in Monopoly:

▪ Si definisce un metodo generale (landedOn) nella superclasse PropertySquare (la classe "Casella Proprietà" che è genitore di Lot, Railroad, Utility). Questo metodo landedOn è il "metodo template".

▪ Il metodo landedOn in PropertySquare contiene i passi comuni a tutte le proprietà (es. tentare l'acquisto, pagare l'affitto).

▪ Per i passi che variano (come il calcolo dell'affitto), si definisce un metodo astratto chiamato getRent (ottenere affitto) sempre nella superclasse PropertySquare.

▪ Ogni sottoclasse specifica (LotSquare, RailRoadSquare, UtilitySquare) ridefinisce (override) il metodo getRent con la sua logica di calcolo dell'affitto unica.

◦ Benefici: Questo riduce la duplicazione del codice e promuove il riuso. La superclasse ("Casella Proprietà") controlla il flusso generale, e lascia alle sottoclassi (Lot, Railroad, Utility) il compito di implementare solo i dettagli specifici.
4.

- Differenza con il Pattern "Strategy":
- Abbiamo già visto che il polimorfismo per landedOn era stato descritto come applicazione del pattern "Strategy".

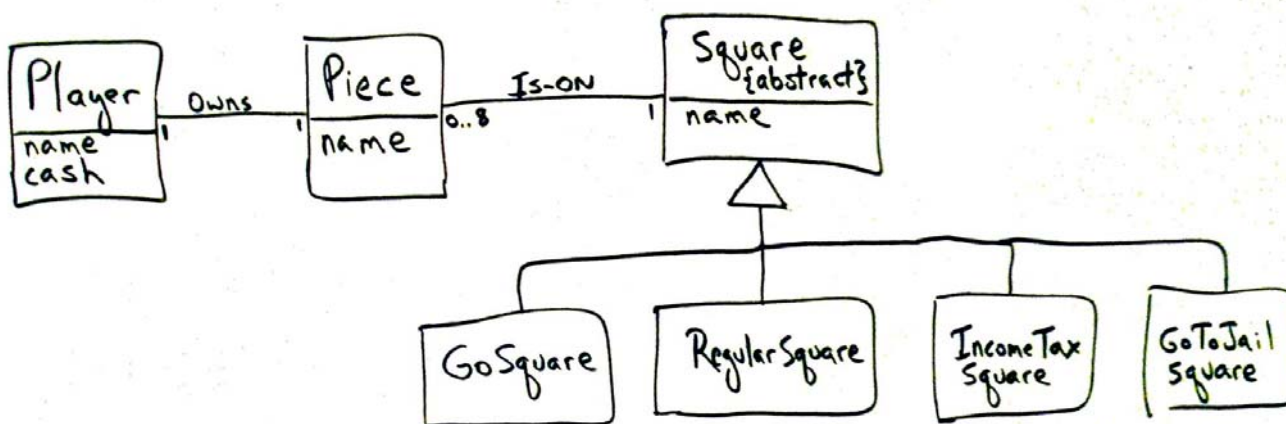
◦ Le note spiegano che:

▪ Template Method usa l'ereditarietà per variare parti di un algoritmo fisso (è meno flessibile se cambia tutta la struttura, ma ottimo per ridurre duplicazioni).

▪ Strategy usa la delega (passare un oggetto che esegue l'azione) per variare l'intero algoritmo (più flessibile per cambiare completamente un comportamento, ma talvolta più complesso).
- In sintesi, il "Template Method" aiuta a gestire algoritmi simili con variazioni locali in modo pulito e senza ripetizioni, incapsulando la logica comune nella superclasse e delegando i dettagli alle sottoclassi
-
- Figura 39.30 Metodi getRent polimorfi.
-
- Figura 39.31 DCD parziale per l'iterazione 3 di Monopoly.

A P S Raffinamenti per Monopoly

Raffinamenti al modello di dominio per l'iterazione 2

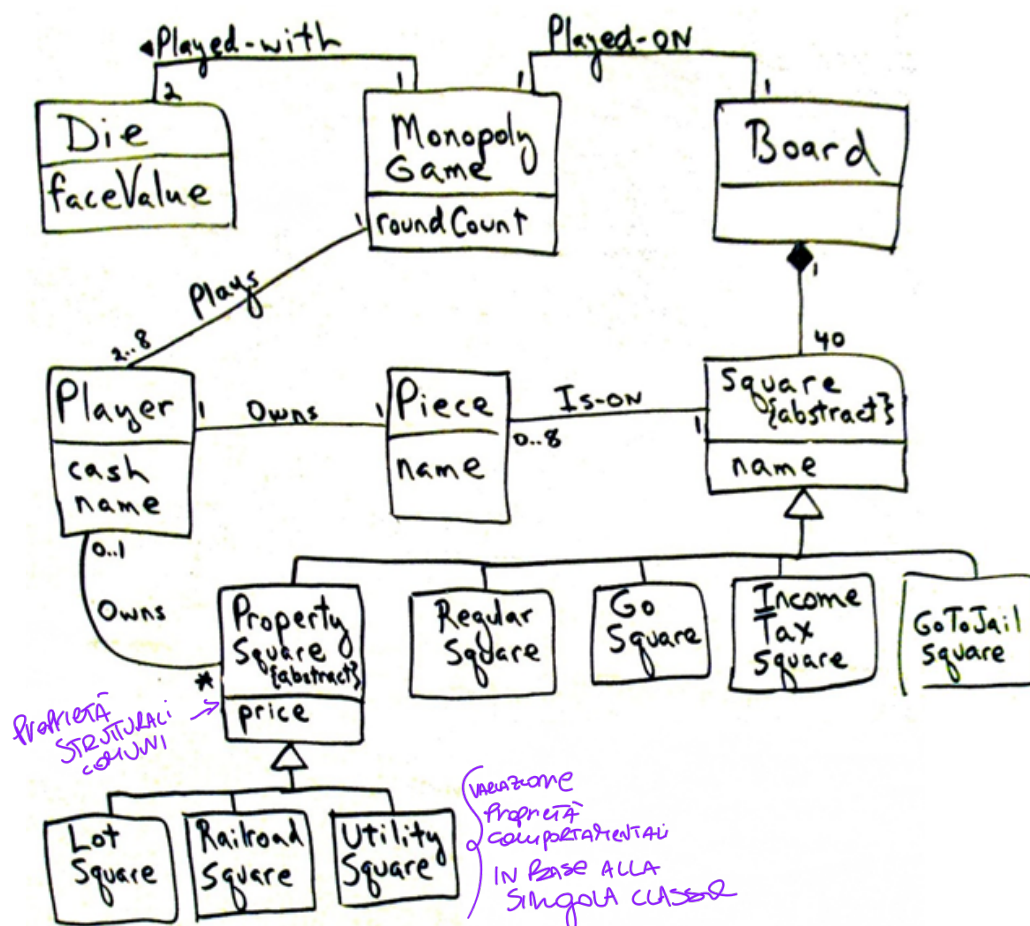


51

Ulteriore modellazione di dominio

Luca Cabibbo - A·P·S

A P S Modello di dominio per l'iterazione 3



52

Ulteriore modellazione di dominio

Luca Cabibbo - A·P·S

* La nuova classe Player

```
public class Player {  
    private String name;  
    private Square location;  
    private int cash;  
  
    private Board board;  
    private Cup cup;  
  
    public Player(String name, Cup cup, Board board) {  
        this.name = name;  
        this.location = board.getStartSquare();  
        this.cash = 1500;  
        this.cup = cup;  
        this.board = board;  
    }  
  
    ... segue ...
```

* La nuova classe Player

```
    public String getName() { return name; }  
    public void setLocation(Square location) {  
        this.location = location;  
    }  
    public void addCash(int c) {  
        cash += c;  
    }  
    public void reduceCash(int c) {  
        cash -= c;  
    }  
    public int getNetWorth() {  
        return cash;    // per ora  
    }  
  
    ... segue ...
```

A P S * La nuova classe Player

```
public void takeTurn() {
    cup.roll();
    int rollTotal = cup.getTotal();
    Square oldLoc = location;
    Square newLoc = board.getSquare(oldLoc, rollTotal);
    location = newLoc;
    System.out.println("It's " + this.getName() + "'s turn: ");
    System.out.println("  start: " + oldLoc +
        ", roll total: " + rollTotal +
        ", end: " + newLoc);

    location.landedOn(this);
}
}
```

A P S * La nuova classe Square

```
public abstract class Square {
    private String name;
    private int index;

    public Square(String name, int index) {
        this.name = name;
        this.index = index;
    }

    public String getName() { return name; }
    public int getIndex() { return this.index; }
    public String toString() {
        return "Square #" + (index+1) + ": " + name;
    }

    public abstract void landedOn(Player p);
}
```

* La classe GoSquare

```
public class GoSquare extends Square {  
    public void landedOn(Player p) {  
        p.addCash(200);  
    }  
}
```

* La classe IncomeTaxSquare

```
public class IncomeTaxSquare extends Square {  
    public void landedOn(Player p) {  
        int w = p.getNetWorth();  
        p.reduceCash( min(200, w/10) );  
    }  
  
    private int min(int a, int b) {  
        return (a<b ? a : b);  
    }  
}
```

* La classe GoToJailSquare

```
public class GoToJailSquare extends Square {  
    private Square jail;  
    public void setJail(Square jail) {  
        this.jail = jail;  
    }  
    public void landedOn(Player p) {  
        p.setLocation(jail);  
    }  
}
```

* La classe RegularSquare

```
public class RegularSquare extends Square {  
    public void landedOn(Player p) {  
        /* do nothing */  
    }  
}
```

A P S 39.11 Template Method [GoF]

Si veda il libro

A P S * La classe astratta PropertySquare

```
public abstract class PropertySquare extends Square {  
  
    private Player owner;  
    private int price;  
  
    public PropertySquare(  
        String name, int index, Player owner, int price) {  
        super(name, index);  
        this.owner = owner;  
        this.price = price;  
    }  
  
    ... segue ...
```

* La classe astratta PropertySquare

```
public void landedOn(Player p) {  
    if (this.owner==null) {  
        p.attemptPurchase(this);  
    } else if (this.owner!=p) {  
        this.payRent(p);  
    }  
}
```

```
private void payRent(Player p) {  
    r = this.getRent();  
    p.reduceCash(r);  
    owner.addCash(r);  
}
```

```
protected abstract int getRent();
```

```
... altri metodi, in particolare getOwner e getPrice ...  
}
```

* La classe LotSquare

```
public class LotSquare extends PropertySquare {  
  
    public LotSquare(...) { ... }  
  
    protected int getRent() {  
        return super.getPrice() / 15;  
    }  
  
}
```

* La classe RailroadSquare

```
public class RailroadSquare extends PropertySquare {  
  
    public RailroadSquare(...) { ... }  
  
    protected int getRent() {  
        int railroadCount = super.getOwner().getRailroadCount();  
        return railroadCount * 25;  
    }  
  
}
```

* La classe UtilitySquare

```
public class UtilitySquare extends PropertySquare {  
  
    public UtilitySquare(...) { ... }  
  
    protected int getRent() {  
        Cup cup = Cup.getInstance();  
        cup.roll();  
        return cup.getTotal() * 4;  
    }  
  
}
```